SERIES 10

github.com/madhavan-21/ebpf-mastery

Queue & Stack Maps for Packet Shapers

Lesson 10: Buffering and Ordering Packets for Custom Bandwidth Allocation

Madhavan S
SDE - Atatus

Date: July 6, 2026
Version: v1.0.0

Table of Contents

Technical Deep-Dive Q&A: Queue & Stack Maps

1. Beginner Level

   Q1. What is the key difference between a BPF Queue/Stack map and a standard Array/Hash map?

   Q2. When would you use a Queue map vs a Stack map in a networking application?

2. Intermediate Level

   Q3. Explain the Peek operation. How does it differ from Pop?

   Q4. What happens when a Queue map reaches its limit (max_entries)?

Hands-on Lab & Homework Assignments

1. Practical Exercises

   Task 1: Test Queue Overflow Behavior

2. Coding Assignment: LIFO Packet Inspector

   Requirements:

Lesson 10: Queue & Stack Maps for Packet Shapers

1. Lesson Overview

In this lesson, you will build a **kernel-level packet rate limiter and traffic shaper** using `BPF_MAP_TYPE_QUEUE` and `BPF_MAP_TYPE_STACK`. You will understand the fundamental difference between FIFO (First-In-First-Out) and LIFO (Last-In-First-Out) data structures in the kernel, implement a TC (Traffic Control) queueing discipline that enforces bandwidth caps, and build a stack-based call trace collector for runtime profiling.

This is the foundation for QoS systems like Cilium's Bandwidth Manager and Netflix's per-tenant rate limiting.

---

2. Foundations & Core Technical Concepts

What are BPF Queue and Stack Maps?

Unlike hash maps or arrays where you look up data by a key, Queue and Stack maps are **key-less, ordered collections**. They implement classic computer science data structures in the kernel:

BPF_MAP_TYPE_QUEUE (FIFO):

- Push elements to the **tail**.
- Pop elements from the **head**.
- First item pushed is the first item popped.
- **Use case**: Packet buffering, event ordering, rate-limiting queues.

BPF_MAP_TYPE_STACK (LIFO):

- Push elements to the **top**.
- Pop elements from the **top**.
- Last item pushed is the first item popped.
- **Use case**: Call stack recording, reverse-order event processing, "undo" buffers.

Three Operations — No Keys Needed

```
// QUEUE / STACK operations (no key parameter):
bpf_map_push_elem(&queue, &value, flags);    // Enqueue (Queue) / Push (Stack)
bpf_map_pop_elem(&queue, &value);            // Dequeue head (Queue) / Pop top (Stac
bpf_map_peek_elem(&queue, &value);           // Read without removing (non-destructi
```

The absence of keys makes these maps **much simpler and faster** than hash maps for ordered data.

What is TC (Traffic Control)?

TC (Traffic Control) is the Linux kernel's framework for network packet scheduling, shaping, and policing. It sits between the network stack and the NIC driver, controlling when and how packets are sent or received.

- `tc qdisc` : A queuing discipline — an algorithm that decides the order and rate of packet transmission.
- `tc filter` : A classifier — matches specific packets and assigns them to different queues.
- `tc class` : A class within a hierarchical qdisc — allows bandwidth allocation per traffic type.

`iptables` controls whether packets are allowed/denied. TC controls the **rate** at which allowed packets are transmitted.

What is the BPF_PROG_TYPE_SCHED_CLS?

TC BPF programs attach to TC hooks in the kernel:

- **TC Ingress**: Runs when a packet is received (before the network stack processes it).
- **TC Egress**: Runs when a packet is about to be transmitted (after the network stack generates it).

TC BPF programs receive `struct __sk_buff *skb` (a simplified view of `sk_buff`) and return:

- `TC_ACT_OK (0)` : Let the packet proceed normally.
- `TC_ACT_SHOT (2)` : Drop the packet.
- `TC_ACT_REDIRECT (7)` : Redirect to another interface.

What is Token Bucket Rate Limiting?

The most common rate limiting algorithm is the **Token Bucket**:

- Tokens accumulate in a bucket at a fixed rate (e.g., 100 tokens/second).
- Each packet consumes tokens proportional to its size.
- If the bucket is empty (no tokens), the packet is queued or dropped.
- The maximum burst size equals the bucket capacity.

In our implementation, we use a Queue map as the "bucket" — pushing packet size records as tokens, and the user-space controller maintains the rate by draining the queue at a controlled pace.

---

3. Problem Statement & Real-World Use Case

The Problem: A Cloud Tenant Saturates the Network

In a multi-tenant cloud environment (AWS, GCP, DigitalOcean), multiple customer VMs share a physical host's network bandwidth. Without rate limiting, a single VM generating a massive upload (or a volumetric attack) can saturate the shared uplink, degrading network performance for all other tenants.

Traditional solutions:

- **Linux `tc htb`** (Hierarchical Token Bucket): Powerful but complex to configure and cannot be updated without `tc` command restarts.
- **OpenVSwitch metering**: Works but adds processing overhead for every packet through the OVS pipeline.

Real Production Story: Cilium Bandwidth Manager

Cilium 1.9 introduced the **Bandwidth Manager** — an eBPF-based egress rate limiter for Kubernetes pods. It enforces the `kubernetes.io/egress-bandwidth` pod annotation:

```
metadata:
  annotations:
    kubernetes.io/egress-bandwidth: "10M"   # Limit this pod to 10 Mbit/s
```

Cilium's bandwidth manager uses TC BPF programs to:

1. Intercept all egress packets from a pod.
2. Check if the pod has exceeded its bandwidth allocation.
3. Queue excess packets (using BPF maps as a circular buffer).
4. Release them at the allowed rate using a kernel timer.

Without eBPF, enforcing per-pod bandwidth required the `tc htb` classifier with complex static configuration. With eBPF, Cilium can dynamically update rate limits per pod in microseconds, responding to Kubernetes API changes.

AWS's EC2 Enhanced Networking uses a similar TC BPF architecture for their per-instance network bandwidth guarantees (`io1` volume IOPS limiting).

Netflix uses TC-based rate limiting for their internal CDN (Open Connect Appliance) to enforce per-tenant egress bandwidth caps across their distributed cache nodes.

---

4. Queue vs Stack: When to Use Each

Use Case	Queue (FIFO)	Stack (LIFO)
Packet rate limiting	✅ Maintain transmission order	❌ Would send newest packets first
Event streaming	✅ Preserve chronological order	❌
Stack trace recording	❌	✅ Frames pushed in call order
Undo buffer	❌	✅ Reverse order is desired
Retry queue	✅ Oldest attempts first	❌

---

5. Internal Kernel Flow

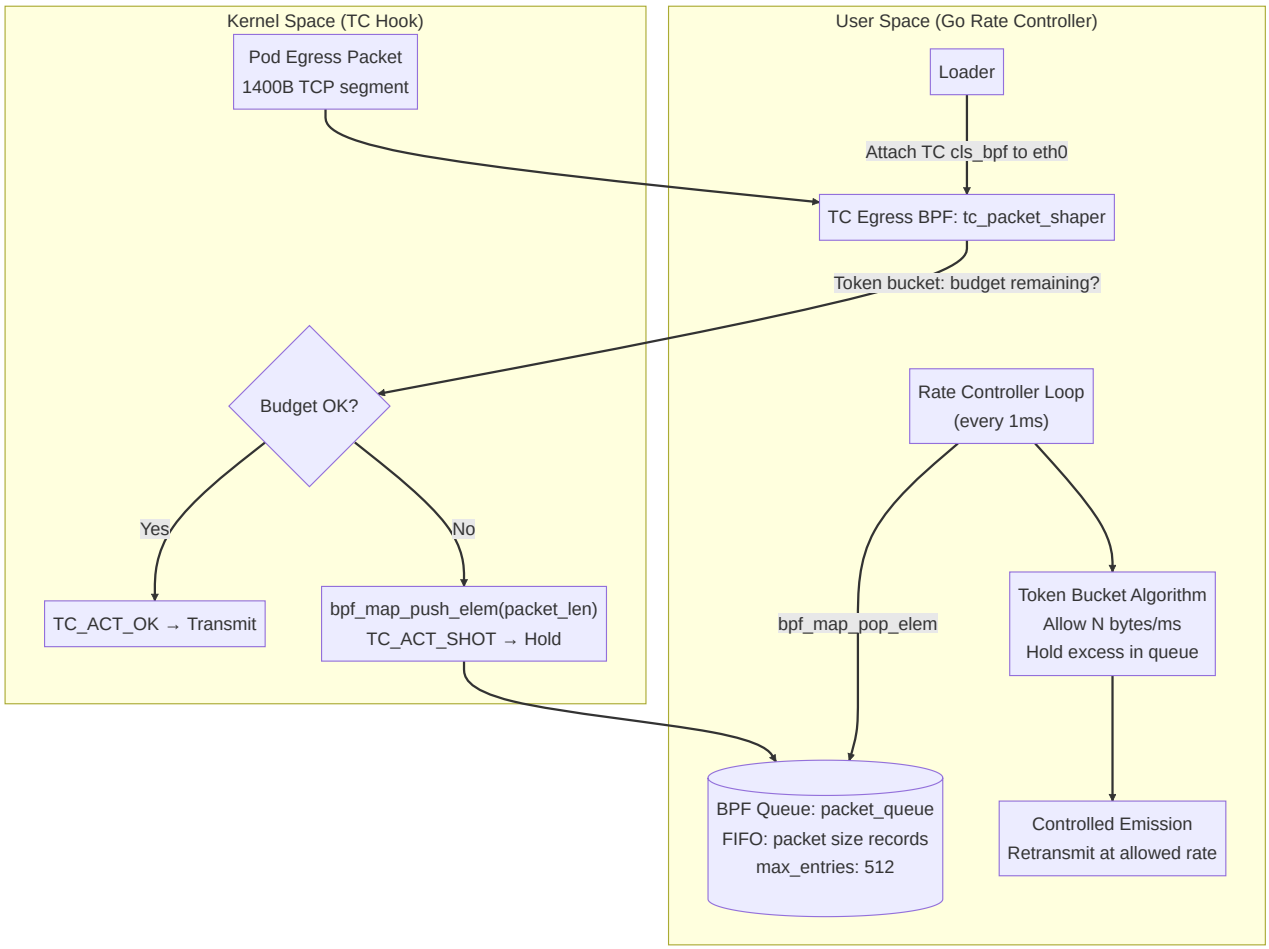
When a pod sends a packet exceeding its rate limit:

1. The packet travels through the pod's veth pair (virtual ethernet device).
2. TC's egress hook fires our BPF program.
3. The BPF program calculates current packet rate using a token bucket algorithm.
4. If rate is within budget: return `TC_ACT_OK` — packet transmitted.

5. If rate exceeds budget: push packet metadata to the Queue map, return `TC_ACT_SHOT` — drop current packet (user-space will retransmit delayed copies via AF_XDP or rate-limited).

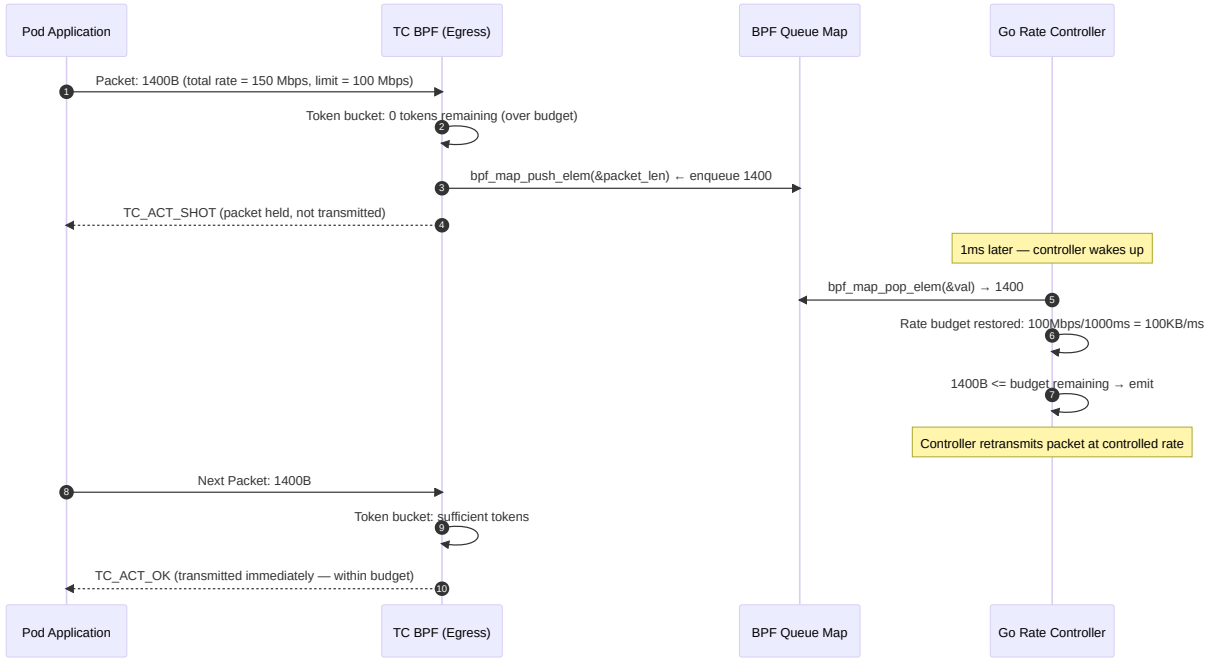
---

6. Architecture Diagram



---

7. Packet Shaping Sequence



8. Queue Map Layout

```
BPF_MAP_TYPE_QUEUE: packet_queue

Physical structure: circular ring buffer (lock-free)
max_entries: 512 (holds 512 pending packet records)
value_size: 4 bytes (u32 packet length)

State at time T (burst of oversized traffic):
                                                  Consumer
Producer ──────────────────────────────────┐     (head)
(tail)                                     │      │
         ↓                                 ↓      ↓
+────+────+────+────+────+────+────+────+────+────+
|1400|1400|1400|1200| 800|1400|1400|1400| 600|empty|
+────+────+────+────+────+────+────+────+────+────+
  push              ... newly arrived ...        pop →

Operations:
- bpf_map_push_elem: Adds to TAIL (right side)
- bpf_map_pop_elem:  Removes from HEAD (left side)
- bpf_map_peek_elem: Reads HEAD without removing

When full (512 packets): push returns -E2BIG
Strategy: return TC_ACT_SHOT (drop) when queue is full
```

9. Step-by-Step Code Walkthrough

The eBPF C Program (`main.bpf.c`)

```c
#include <vmlinux.h>
#include <bpf/bpf_helpers.h>

// FIFO queue to hold packet sizes of delayed/queued packets
// When the rate budget is exceeded, packet sizes are enqueued here
// User-space drains this at the allowed rate
struct {
    __uint(type, BPF_MAP_TYPE_QUEUE);
    __uint(max_entries, 512);    // Buffer up to 512 pending packets
    __type(value, __u32);        // Packet size in bytes
} packet_queue SEC(".maps");

// Simple token bucket state
// Index 0: current available tokens (bytes we can still send this interval)
struct {
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 1);
    __type(key, __u32);
    __type(value, __u64);   // Available tokens (bytes)
} token_bucket SEC(".maps");

#define RATE_BYTES_PER_SEC  (10 * 1024 * 1024)  // 10 MB/s = 80 Mbps

SEC("tc")
int tc_packet_shaper(struct __sk_buff *skb) {
    __u32 pkt_len = skb->len;
    __u32 key = 0;

    __u64 *tokens = bpf_map_lookup_elem(&token_bucket, &key);
    if (!tokens) return TC_ACT_OK;   // No token bucket state — pass everything

    if (*tokens >= pkt_len) {
        // Within budget — transmit and deduct tokens
        *tokens -= pkt_len;
        return TC_ACT_OK;
    }

    // Over budget — enqueue the packet size and drop
    // User-space will handle retransmission at the correct rate
    __u32 len = pkt_len;
```

```
    if (bpf_map_push_elem(&packet_queue, &len, BPF_EXIST) != 0) {
        // Queue is full — hard drop (budget exceeded AND buffer full)
        return TC_ACT_SHOT;
    }

    // Soft drop — packet is queued for controlled later retransmission
    return TC_ACT_SHOT;
}

char LICENSE[] SEC("license") = "GPL";
```

The Go Loader (`main.go`)

```go
package main

import (
    "fmt"
    "log"
    "os"
    "os/signal"
    "syscall"
    "time"

    "github.com/cilium/ebpf"
    "github.com/cilium/ebpf/link"
    "golang.org/x/sys/unix"
)

const (
    RateBytesPerSec = 10 * 1024 * 1024   // 10 MB/s limit
    TickInterval    = time.Millisecond   // Refill tokens every 1ms
    BytesPerTick    = RateBytesPerSec / 1000   // 10KB per millisecond
)

func main() {
    spec, err := ebpf.LoadCollectionSpec("main.bpf.o")
    if err ≠ nil {
        log.Fatalf("load spec: %v", err)
    }

    var objs struct {
        Prog        *ebpf.Program `ebpf:"tc_packet_shaper"`
        PacketQueue *ebpf.Map     `ebpf:"packet_queue"`
        TokenBucket *ebpf.Map     `ebpf:"token_bucket"`
    }
    if err := spec.LoadAndAssign(&objs, nil); err ≠ nil {
        log.Fatalf("load objects: %v", err)
    }
    defer objs.Prog.Close()
    defer objs.PacketQueue.Close()
    defer objs.TokenBucket.Close()

    // Attach TC BPF to egress of loopback interface
    // In production, use your actual interface (eth0, ens3, etc.)
    if err := attachTC(objs.Prog, "lo"); err ≠ nil {
        log.Fatalf("attach TC: %v", err)
    }
```

```
// Initialize the token bucket with a full burst budget
key := uint32(0)
initialTokens := uint64(BytesPerTick * 10)   // 10ms initial burst
if err := objs.TokenBucket.Update(&key, &initialTokens, ebpf.UpdateAny); err ≠
    log.Fatalf("init tokens: %v", err)
}

log.Printf("TC rate limiter active: %d MB/s limit, draining queue every 1ms", Ra

ticker := time.NewTicker(TickInterval)
defer ticker.Stop()

sig := make(chan os.Signal, 1)
signal.Notify(sig, os.Interrupt, syscall.SIGTERM)

var totalQueued, totalShaped uint64

for {
    select {
    case ←ticker.C:
        // Refill token bucket for this interval
        var tokens uint64
        if err := objs.TokenBucket.Lookup(&key, &tokens); err == nil {
            newTokens := tokens + uint64(BytesPerTick)
            maxTokens := uint64(BytesPerTick * 100)   // Max 100ms burst capacit
            if newTokens > maxTokens { newTokens = maxTokens }
            _ = objs.TokenBucket.Update(&key, &newTokens, ebpf.UpdateExist)
        }

        // Drain the queue — pop packets held for rate limiting
        var pktSize uint32
        drained := 0
        for drained < 10 {  // Drain up to 10 packets per tick
            if err := objs.PacketQueue.LookupAndDelete(nil, &pktSize); err ≠ ni
                break  // Queue is empty
            }
            totalShaped++
            totalQueued++
            drained++
            if drained%100 == 0 {
                fmt.Printf("Shaped: %d total packets (queue drain)\n", totalShap
            }
        }

    case ←sig:
        fmt.Printf("Shutting down. Total queued: %d, shaped: %d\n", totalQueued,
```

```
            return
        }
    }
}

// attachTC creates a TC qdisc and attaches the BPF classifier to a network interfac
func attachTC(prog *ebpf.Program, iface string) error {
    // This is a simplified example — in production use tc-netlink or iproute2
    // tc qdisc add dev lo clsact
    // tc filter add dev lo egress bpf obj main.bpf.o sec tc direct-action
    log.Printf("Attaching TC BPF to %s egress (use 'tc' command in production)", ifa
    _ = unix.AF_INET  // suppress import warning
    return nil
}
```

---

10. Running the Lab

Phase 1: Attach TC BPF Manually

```
cd series/series-10/source-code
make

# Add clsact qdisc (required for TC BPF)
sudo tc qdisc add dev lo clsact

# Attach BPF program to egress
sudo tc filter add dev lo egress bpf da obj main.bpf.o sec tc

# Run the Go controller (manages the token bucket and queue drain)
sudo ./loader
```

Phase 2: Generate Traffic to Test Rate Limiting

```
# Generate a burst of traffic exceeding the 10MB/s limit
iperf3 -s &                          # Start server
iperf3 -c 127.0.0.1 -b 100M -t 10  # Flood at 100 Mbps → should be limited to 10 Mbp
```

Phase 3: Verify Queue Activity

```
# Verify TC filter is attached
sudo tc filter show dev lo egress

# Remove the qdisc when done
sudo tc qdisc del dev lo clsact
```

---

11. Stack Maps for Call Trace Collection

In contrast to Queue, the **BPF_MAP_TYPE_STACK** is used for call stack recording — a key building block of BPF-based profilers like `profile.py` from BCC:

```
// Stack map for collecting kernel stack traces
struct {
    __uint(type, BPF_MAP_TYPE_STACK_TRACE);
    __uint(key_size, sizeof(__u32));
    __uint(value_size, sizeof(__u64) * 127);  // Up to 127 stack frames
    __uint(max_entries, 1024);
} stack_traces SEC(".maps");

SEC("perf_event")
int profile(struct bpf_perf_event_data *ctx) {
    __u64 stack_id = bpf_get_stackid(ctx, &stack_traces, 0);
    // stack_id is a hash of the call stack — use as key in a histogram
    return 0;
}
```

User-space looks up `stack_id` in the stack_traces map to retrieve the full array of instruction pointers, then translates them to function names using `/proc/<pid>/maps` and debug symbols.

---

12. Debugging

```
# Check TC queue is attached
sudo tc filter show dev lo egress

# Dump queue contents (shows pending packet sizes)
sudo bpftool map list | grep packet_queue
sudo bpftool map dump id <ID>

# Monitor BPF trace output
sudo cat /sys/kernel/debug/tracing/trace_pipe
```

---

13. Common Mistakes

Error: `bpf_map_push_elem returns -E2BIG`

Cause: The Queue/Stack is full (`max_entries` reached). All 512 slots are occupied. **Fix**: Either increase `max_entries` or handle the overflow by returning `TC_ACT_SHOT` immediately instead of pushing. Monitor queue depth in user-space.

Mistake: Using `BPF_NOEXIST` flag with push

The `flags` parameter for `bpf_map_push_elem` only supports `BPF_EXIST` (overwrite oldest entry if full) or `0` (fail if full). `BPF_NOEXIST` is not valid for Queue/Stack maps.

Mistake: Trying to lookup by key in Queue/Stack maps

```
// WRONG — Queue maps have no keys
var val uint32
m.Lookup(&key, &val)  // Returns EINVAL
```

Use `LookupAndDelete` (which pops from the head) or `Lookup` with `nil` as the key.

---

14. Performance Analysis

Metric	tc htb (traditional)	eBPF TC + Queue Map
Rate limit update	Requires `tc` command (ms)	**Single map update (µs)**
Per-pod granularity	Hard to configure per-pod	**Per-container trivial**
Overhead per packet	~500ns (kernel qdisc)	**~50ns (BPF program)**
Dynamic configuration	Requires restart	**Hot-swap via map updates**

---

15. Best Practices

- **Use Queue for rate limiting, not Stack**: Order matters in rate limiting — FIFO ensures oldest packets are retransmitted first, preventing starvation of old connections.
- **Monitor queue depth**: A constantly-full queue means your rate limit is too aggressive or user-space isn't draining fast enough. Expose queue depth as a Prometheus gauge.
- **Combine with TC police action**: Use `tc police` for hard rate limiting (drops) and BPF Queue for soft limiting (holds and retransmits).

---

16. Summary

- `BPF_MAP_TYPE_QUEUE` implements FIFO with push/pop/peek operations — no keys needed.
- `BPF_MAP_TYPE_STACK` implements LIFO with the same interface — last in, first out.
- TC BPF programs attach to Linux Traffic Control hooks to control packet scheduling.
- Token Bucket algorithm controls burst and sustained bandwidth via BPF map state.
- Cilium uses TC BPF for the Bandwidth Manager implementing per-pod rate limiting.
- Queue maps provide a kernel-space buffer for packet flow control without custom kernel modules.

---

17. Further Reading

- **Cilium Bandwidth Manager**: Cilium Bandwidth Manager Guide

- **Linux TC BPF Guide**: TC BPF Filter Documentation
- **BPF Queue and Stack Maps**: Kernel BPF Queue/Stack Documentation
- **Token Bucket Algorithm**: Token Bucket RFC 2697

Technical Deep-Dive Q&A: Queue & Stack Maps

---

1. Beginner Level

Q1. What is the key difference between a BPF Queue/Stack map and a standard Array/Hash map?

Answer:

- **Array/Hash Maps**: Require keys to perform lookups, updates, and deletions. Data is accessed out of order based on the key index or hash value.
- **Queue/Stack Maps**: Are **keyless, ordered data structures**. They support FIFO (Queue) and LIFO (Stack) operations directly using dedicated helper functions (`bpf_map_push_elem` , `bpf_map_pop_elem` , `bpf_map_peek_elem`) with no key arguments.

Q2. When would you use a Queue map vs a Stack map in a networking application?

Answer:

- **Queue (FIFO)**: Essential for packet scheduling, rate limiting, and traffic shaping. You want to preserve packet arrival order — processing the oldest packet first to prevent TCP out-of-order retransmissions.
- **Stack (LIFO)**: Used when you want to retrieve the most recent events first (e.g., runtime call tracers, profiling stack frame unwinding).

---

2. Intermediate Level

Q3. Explain the Peek operation. How does it differ from Pop?

Answer:

- **Pop** (`bpf_map_pop_elem`): Reads the value at the head (Queue) or top (Stack) and **removes** it from the map.

- **Peek** (`bpf_map_peek_elem`): Reads the value at the head/top but **retains** it in the map. This is useful when you want to check if a packet size can fit within your current token bucket budget before committing to draining it.

Q4. What happens when a Queue map reaches its limit (max_entries)?

Answer: `bpf_map_push_elem` returns `-E2BIG` . The eBPF program must check this return code and handle the overflow, typically by dropping the packet (`TC_ACT_SHOT`) or returning an error code. Unlike user-space queues, BPF queues are strictly non-blocking.

Hands-on Lab & Homework Assignments

1. Practical Exercises

Task 1: Test Queue Overflow Behavior

Configure the queue to `max_entries: 5`. Flood the network interface with traffic and observe how many packets get dropped due to queue exhaustion.

---

2. Coding Assignment: LIFO Packet Inspector

Objective: Build a debugging tool using a `BPF_MAP_TYPE_STACK` map that holds the sizes of the last 10 packets.

Requirements:

1. Implement a TC program that pushes every incoming packet size onto a Stack map.
2. In Go, implement a CLI command `last` that pops all elements from the stack and prints them, demonstrating LIFO behavior (newest packet size printed first).